CLOUD COMPUTING
MINI PROJECT
Release: V1.0.0[final]

NAME: YASH CHAHAR
ROLLNO: 230103020
COURSE: CLOUD COMPUTING

GitHub: https://github.com/yograjchahar05/cc-assignment.git

1. What I learned

In this project I learned how to deploy a machine learning model using cloud-native techniques. I used knowledge from a large teacher model (Phi-3.5, 3.8B parameters) to a smaller student model (Pythia - 70M).

I learned that direct logit based distillation was difficult was because both model used different tokenizers and the teacher model provided only text output through an API. Therefore, I used sequence-level-distillation where the Student learned from the teacher's generated JSON responses.

2. Technical challenges and solutions.

The biggest challenges was that the training loss became NaN during CPU Training. After debugging, I found that FP16 precision caused numerical overflow.

I solved this problem by changing the model to FP32 (32-bit precision). I also used dynamic padding to reduce unnecessary computation, improving batch time from 25 second to 1.4 seconds.

- Epoch 1: 0.6368
- Epoch 2: 0.1128
- Epoch 3: 0.0523

3. Conclusion:

This project improved my understanding of Machine learning, Knowledge Distillation, cloud deployment, and debugging. I also learned how to identify problems and find practical solutions.

⇨ Microservices and Setup | Startup Management

In this task, I learned how to deploy a FastAPI application using Docker and Kubernetes. Instead of storing model weights inside the Docker image, I stored them in SeaweedFS S3 and loaded them when the application started. I also implemented a readiness probe (/readyz) to ensure that kubernetes sends requests only when the model is fully loaded.

In testing, all 50 request were successful, with an average response of 444.7 ms. Memory usage remained within the 1GB limit.

⇨ Networking and Firewall Rules.

In Task 5, I learned about VLSM subnetting and Network Access Control Lists (NACLs).
I understood that NACLs are stateless, meaning seperate rules are needed for incoming and outgoing traffic.
I also learned that firewall rules are checked in numerical order, so deny rules must come before general allow rules.

I tested my NACL configuration using nacl_sim.py and achieved 6/6 correct packet decisions.

Finally, I learned that running kubernetes on a laptop cannot provide real hardware fault tolerance.
Production system need multiple availability zones for better reliability.

CI/CD Pipeline and Model Training.

In Task 6, I created a 5-stage GitHub Actions pipeline to check quality, run tests and validate model performance. I learned that passing software test does not always mean a machine learning model works correctly therefore. I added a quality check requiring at least 95% valid JSON output.

The untrained model scored 0%, so the pipeline failed. the trained model score 100%. successfully passing the quality check.

Cloud Cost And Performances:

In task 7, I learned how to calculated cloud hosting costs and monitor system performance.

The cost of serving 1,000 requests was approximately $0.46. I also achieved 100% successful requests during testing with a p95 response time of 2.2 seconds.

This taught me the importance of controlling cloud costs scaling resources, and maintaining service reliability.

What Could be Improved

There are a few areas where this project can be improved.

1. Faster Inference: Use ONNX Runtime and INT8 quantization to improve speed.
2. Faster Training Data Generation: Use parallel processing to generate data more quickly.
3. Better Accuracy: Use a larger student model to improve intent recognition.

Conclusion: Overall this project helped me gain practical knowledge of Cloud Computing, machine learning. Docker Kubernetes, networking. CI/CD, and cloud cost management. It also improved my problem-solving and debugging skills.